

Fast Realistic Rendering

OpenGL Introduction

OpenGL Pipeline

Initialization

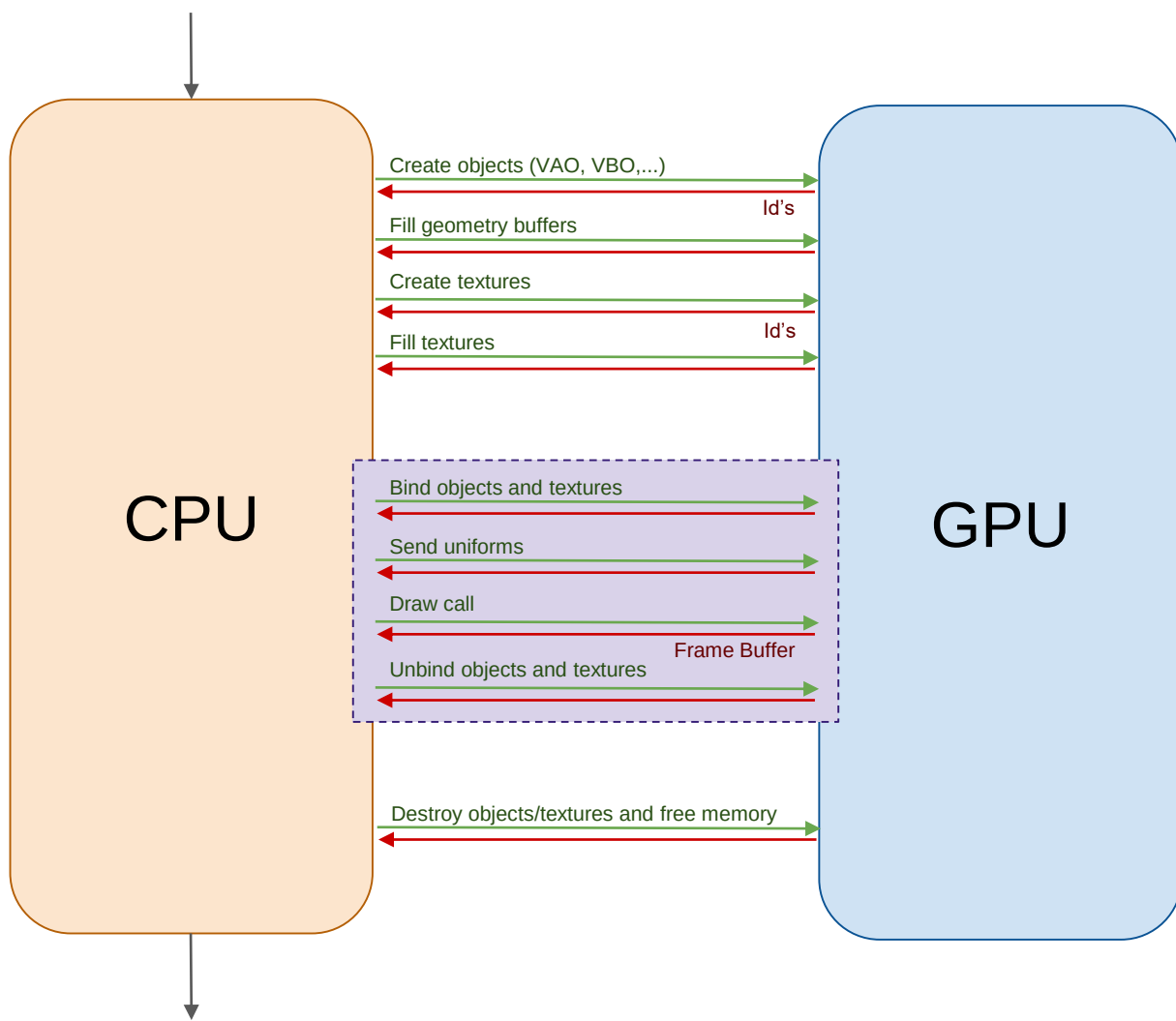
initializeGL()

Rendering
loop

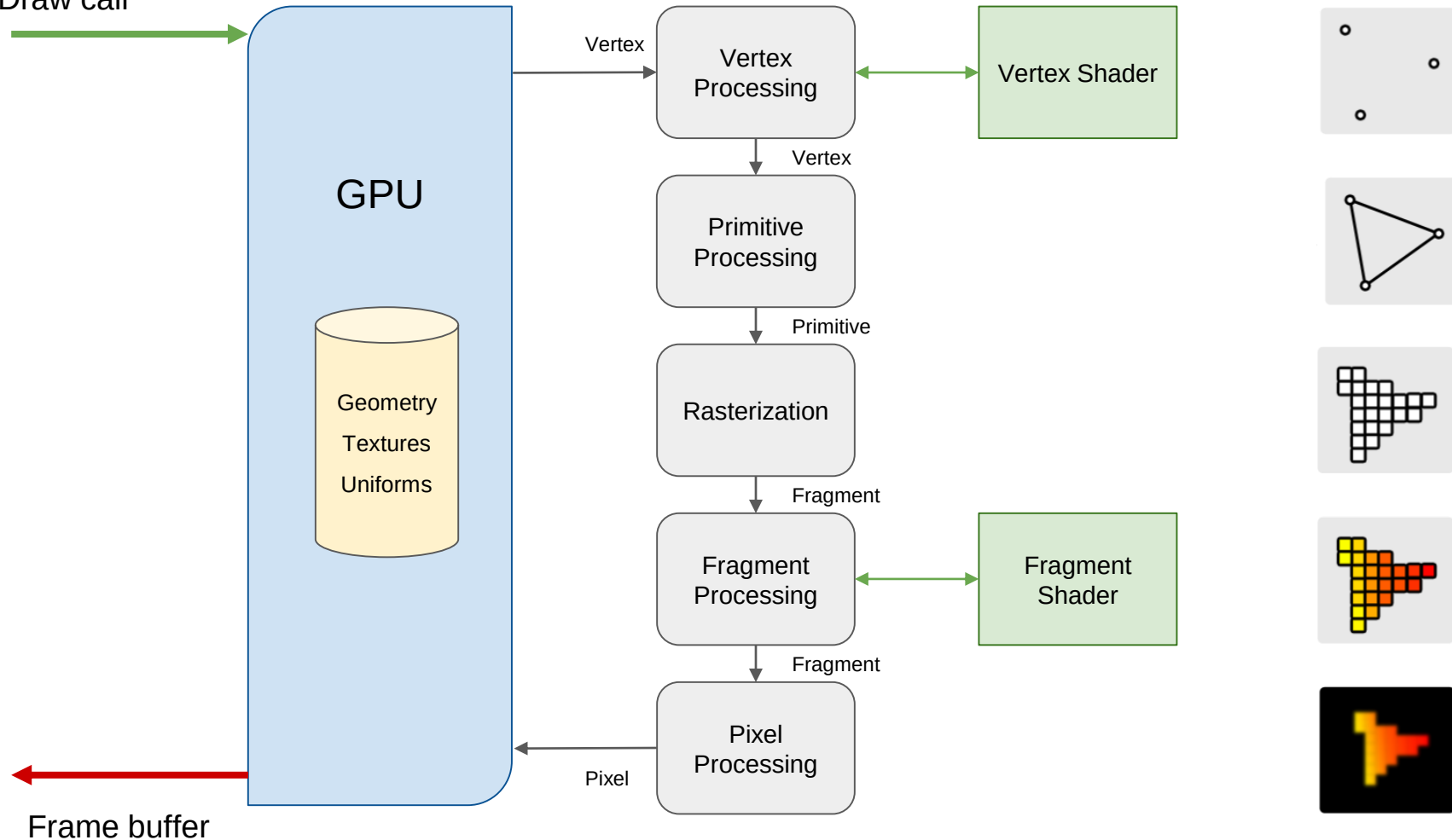
paintGL()

Termination

~()



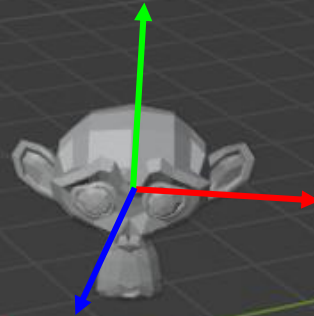
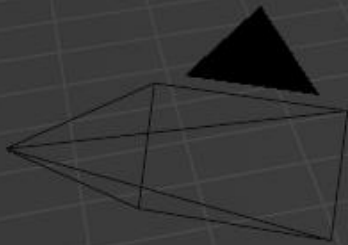
Draw call



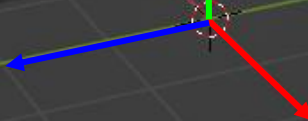
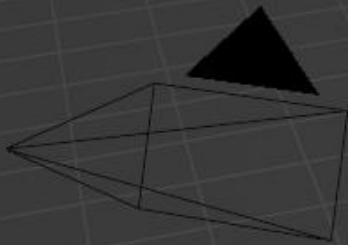
Frame buffer

Coordinate Systems

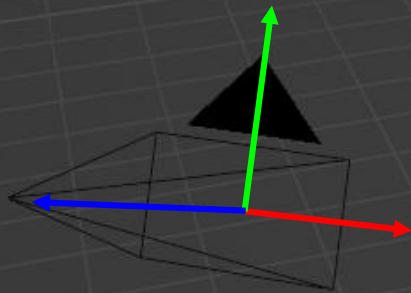
Object Space (OS)



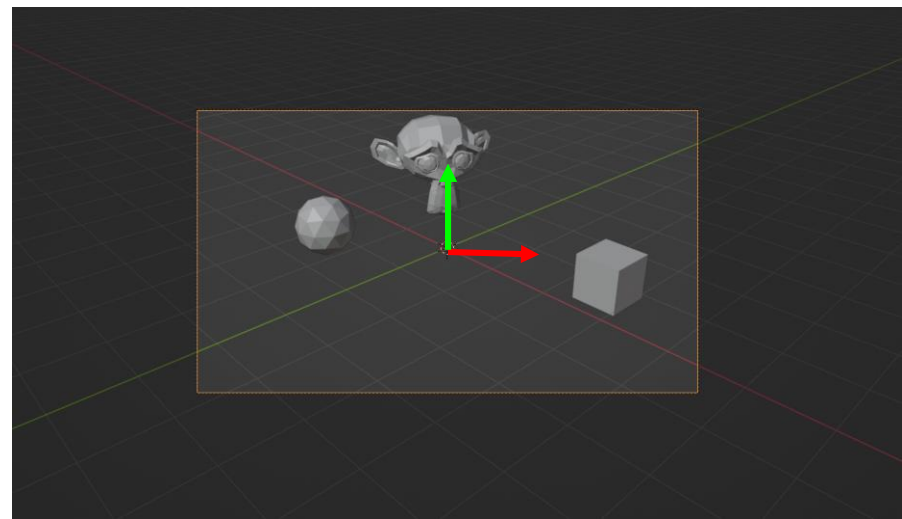
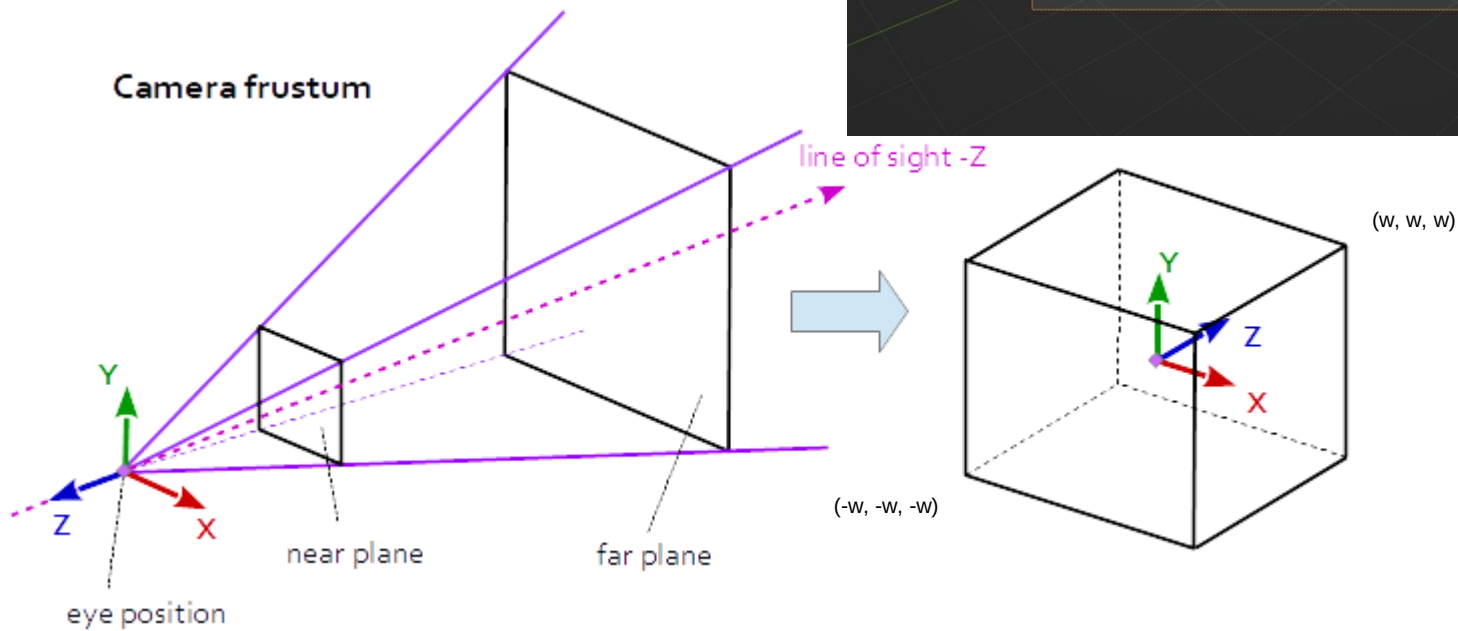
World Space (WS)



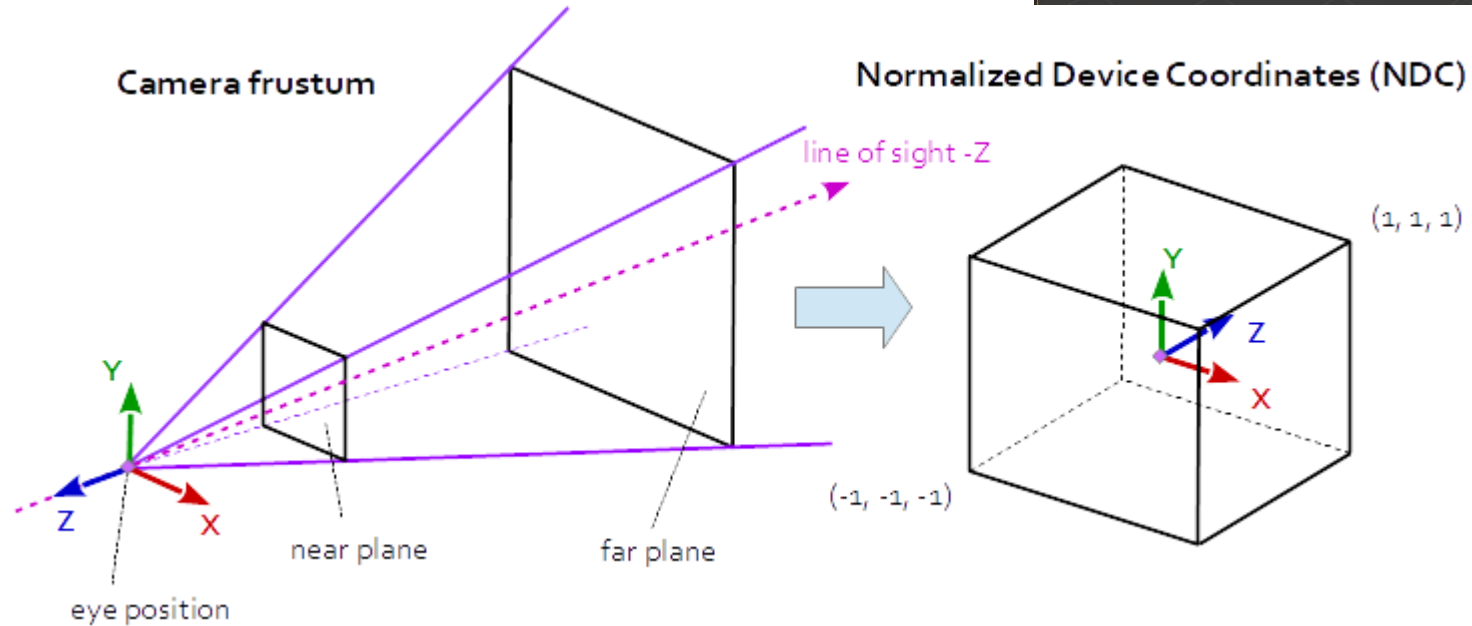
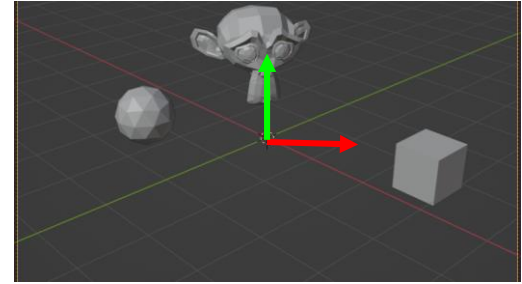
Eye Space (ES)



Clip Space (CS)

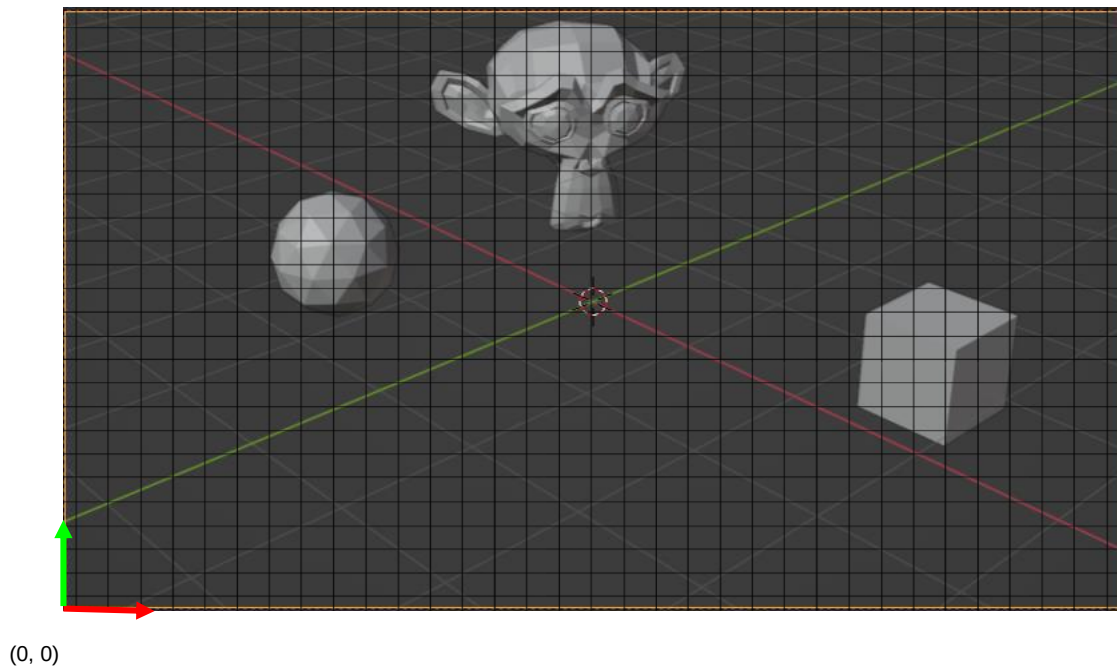


Normalized Device Coordinates (NDC)

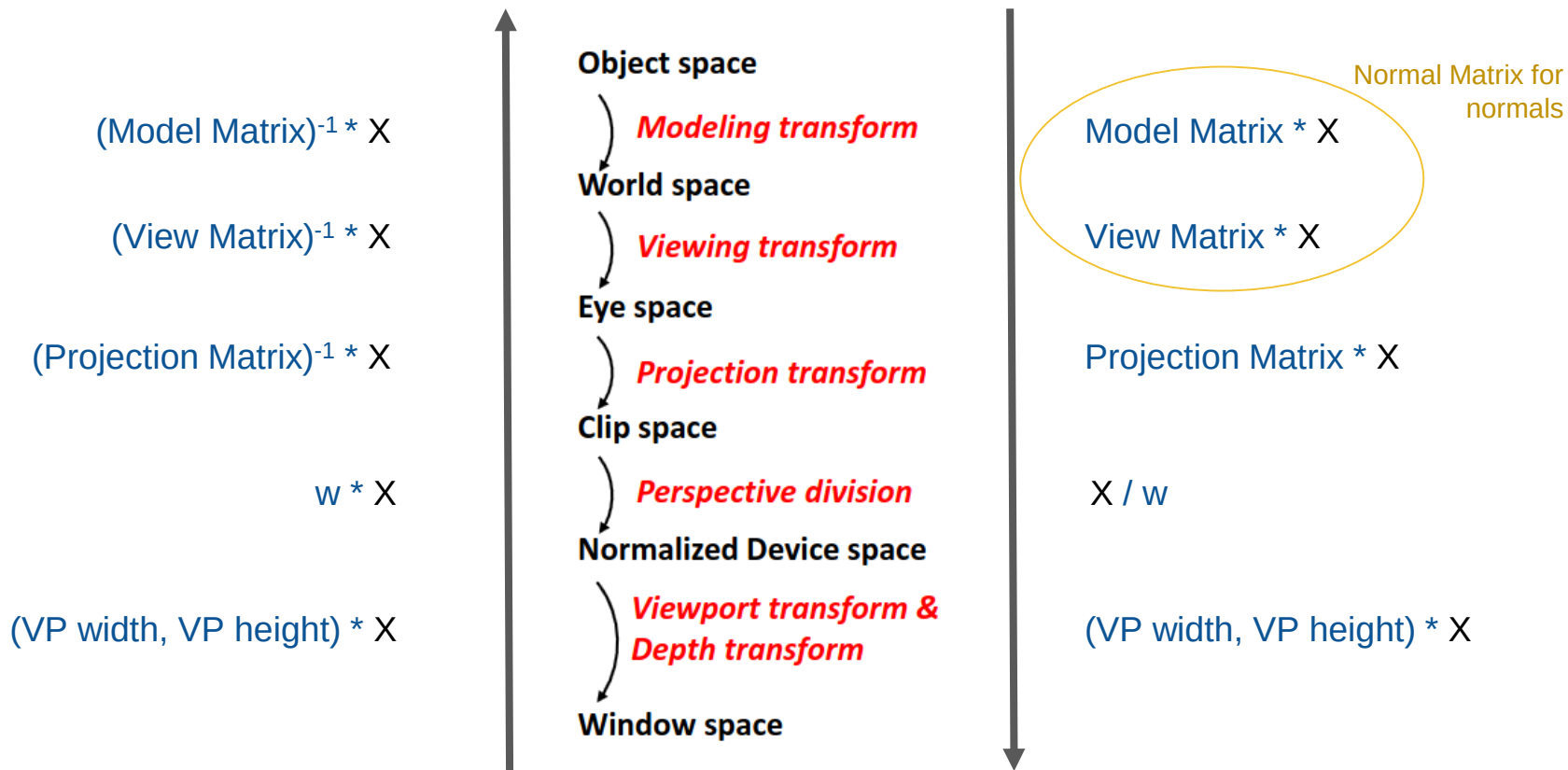


Window Space (WinS)

(VP width, VP height)



Changing between spaces



Vertex and Fragment Shaders

Vertex Shader

- Goal: Process vertices by performing needed computations per vertex
- Input: Geometric attributes
- Input: Uniforms
- Output: `gl_Position` -> position of the vertex in CS (mandatory)
- Output: varying's with information to FS

Fragment Shader

- Goal: Process fragments to obtain the final color of the fragment
- Input: varying's of the VS interpolated
- Input: Uniforms
- Output: Color of the fragment

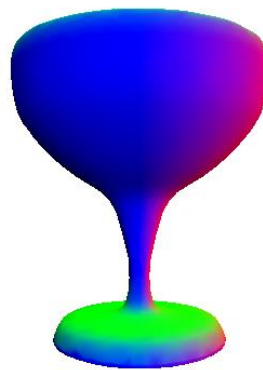
Example: Normal visualization in ES

Vertex Shader

```
1 #version 330 core
2
3 layout (location = 0) in vec3 vertex;
4 layout (location = 1) in vec3 normal;
5 layout (location = 2) in vec3 color;
6 layout (location = 3) in vec2 texCoord;
7
8 out vec3 normal_es;
9
10 uniform mat4 modelViewProjectionMatrix;
11 uniform mat3 normalMatrix;
12
13 void main()
14 {
15     normal_es = normalize(normalMatrix * normal);
16     gl_Position = modelViewProjectionMatrix * vec4(vertex, 1.0);
17 }
```

Fragment Shader

```
1 #version 330 core
2
3 in vec3 normal_es;
4 out vec4 fragColor;
5
6 void main()
7 {
8     fragColor = vec4(normal_es, 1);
9 }
```



VAO and VBO

Vertex Array Object

An object in the GPU memory that encapsulates a 3D object and can handle several attributes

[Initialization]	Creation:	<i>glGenVertexArrays(NumOfArrays, &ID)</i>
[Initialization]	Config attrib:	<i>glVertexAttribPointer(location, NumOfCoords, TypeOfCoords, Normalized, Stride, *Data)</i>
[Initialization]	Enable attrib:	<i>glEnableVertexAttribArray(location)</i>
[Init / Rendering Loop]	Binding:	<i>glBindVertexArray(&ID)</i>
[Init /Rendering Loop]	Unbinding:	<i>glBindVertexArray(0)</i>
[Rendering Loop]	Draw call:	<i>glDrawElements(GL_TRIANGLES, numIndices, type, (GLvoid*)0)</i> <i>glDrawArrays(GL_TRIANGLES, 0, numVertices)</i>
[Termination]	Cleaning:	<i>glDeleteVertexArrays(NumOfArrays, &ID)</i>

Vertex Buffer Object

A buffer in the GPU memory that stores an attribute of a VAO. We need to create as many as attributes we use, and one for the indices.

[Initialization]	Creation:	<code>glGenBuffers(NumOfBuffers, &ID)</code>
[Initialization]	Binding (attrib):	<code>glBindBuffer(GL_ARRAY_BUFFER, &ID)</code>
[Initialization]	Binding (indices):	<code>glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, &ID)</code>
[Initialization]	Filling (attrib):	<code>glBufferData(GL_ARRAY_BUFFER, sizeof(type)*numVerts*numCoords, &data[0], GL_STATIC_DRAW)</code>
[Initialization]	Filling (attrib):	<code>glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(type)*numIndices, &indices[0], GL_STATIC_DRAW)</code>
[Termination]	Cleaning:	<code>glDeleteBuffers(NumOfBuffers, &ID)</code>

Initialization example using indices

```
// Step 1: Create and fill STL vectors(coords, normals...)
vector<float> vertices;    // (x,y,z)
vector<float> normals;    // (nx,ny,nz)
vector<float> colors;     // (r,g,b)
vector<float> texCoords;  // (s,t)
vector<unsigned int> indices; //i0,i1,i2, i3,i4,i5...
for(...) {
    vertices.push_back(x);
    vertices.push_back(y);
    vertices.push_back(z);
for(...) {
    indices.push_back(index);
```

Initialization example using indices

// Step 2: Create VAO & empty VBOs

```
GLuint VAO;
```

```
g.glGenVertexArrays(1,&VAO);
```

```
GLuint coordBufferID;
```

```
g.glGenBuffers(1, &coordBufferID);
```

```
GLuint normalBufferID;
```

```
g.glGenBuffers(1, &normalBufferID);
```

```
...
```

```
GLuint indexBufferID;
```

```
g.glGenBuffers(1, &indexBufferID);
```

Initialization example using indices

```
// Step 3: Define VBO data (coords,normals,indices)
g.glBindVertexArray(VAO);
g.glBindBuffer(GL_ARRAY_BUFFER, coordBufferID);
g.glBufferData(GL_ARRAY_BUFFER, sizeof(float)*vertices.size(), &vertices[0],
    GL_STATIC_DRAW);
g.glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
g.glEnableVertexAttribArray(0);

g.glBindBuffer(GL_ARRAY_BUFFER, normalBufferID);
g.glBufferData(GL_ARRAY_BUFFER, sizeof(float)*normals.size(), &normals[0],
    GL_STATIC_DRAW);
g.glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, 0);
g.glEnableVertexAttribArray(1);
...
g.glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, indexBuffersID);
g.glBufferData(GL_ELEMENT_ARRAY_BUFFER,
    sizeof(int)*indices.size(), &indices[0], GL_STATIC_DRAW);

g.glBindVertexArray(0);
```

Draw call example with indices

```
// Draw a single instance of the 3D model
g.glBindVertexArray(VAO);
g.glDrawElements(GL_TRIANGLES, numIndices, GL_UNSIGNED_INT, (GLvoid*)0);
//numIndices=indices.size()
g.glBindVertexArray(0);
```

```
// Draw multiple instances of the same 3D model
g.glBindVertexArray(VAO);
g.glDrawElementsInstanced(GL_TRIANGLES, numIndices, GL_UNSIGNED_INT,
(GLvoid*)0, numInstances);
g.glBindVertexArray(0);
```

VS: int gl_InstanceID → instance number (0...numInstances-1)

Termination example with indices

// Clean up

```
g.glDeleteBuffers(1, &coordBufferID);
```

```
g.glDeleteBuffers(1, &normalBufferID);
```

```
...
```

```
g.glDeleteBuffers(1, &indexBufferID);
```

```
g.glDeleteVertexArrays(1, &VAO);
```

Textures

Textures

A texture is a buffer that contains data (normally color information) with more detail than surface resolution (primitive size). We can access it per fragment.

[Initialization]	Creation:	<i>glGenTextures(NumOfText, &ID)</i>
[Initialization]	Filling:	<i>glTexImage2D(GL_TEXTURE_2D, MipMapLevel, InternalFormat, width, height, border, Format, type, *Data)</i>
[Initialization]	Filtering config:	<i>glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER / GL_TEXTURE_MIN_FILTER, GL_NEAREST / GL_LINEAR / ...)</i>
[Initialization]	Wrap config:	<i>glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S / GL_TEXTURE_WRAP_G, GL_CLAMP / GL_REPEAT / ...)</i>
[Init / Rendering Loop]	Binding:	<i>glBindTexture(GL_TEXTURE_2D, &ID)</i>
[Init /Rendering Loop]	Unbinding:	<i>glBindTexture(0)</i>
[Rendering Loop]	Filling with FB:	<i>glCopyTexImage2D(GL_TEXTURE_2D, MipMapLevel, InternalFormat, x, y, width, height, border)</i>
[Termination]	Cleaning:	<i>glDeleteTextures(NumOfArrays, &ID)</i>

Example Initialization - Loading a texture from a file Qt

```
// Load Texture 1
QString filename = QFileDialog::getOpenFileName(0, "Open Image", "/assig/grau-g/Textures", "Image file (*.png *.jpg)");»
QImage img0(filename);»
QImage im0 = img0.convertToFormat(QImage::Format_ARGB32).rgbSwapped().mirrored();
    g.makeCurrent();
g.glActiveTexture(GL_TEXTURE0);
g.glGenTextures( 1, &textureId0);
g.glBindTexture(GL_TEXTURE_2D, textureId0);
g.glTexImage2D( GL_TEXTURE_2D, 0, GL_RGB, im0.width(), im0.height(), 0, GL_RGBA, GL_UNSIGNED_BYTE, im0.bits());
g.glTexParameteri( GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR );
g.glTexParameteri( GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR );
g.glTexParameteri( GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT );
g.glTexParameteri( GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT );
g.glBindTexture(GL_TEXTURE_2D, 0);
```

Example Rendering Loop - Configuring texture access in shaders

```
// bind texture
g.glActiveTexture(GL_TEXTURE0);
g.glBindTexture(GL_TEXTURE_2D, textureId0);
// bind shader and define uniforms
program->bind();
program->setUniformValue("sampler0", 0); // texture unit of the texture

//DRAW CALL

// unbind textures
g.glActiveTexture(GL_TEXTURE0);
g.glBindTexture(GL_TEXTURE_2D, 0);
```

Exercices for next week

1. Install the *skeleton* provided and test it (nothing is rendered).
2. Follow the instructions in the document *OpenGL Introduction Session Guide*
 - a) Render the sphere
 - b) Render the skybox
 - c) Render the sphere textured
 - d) Render the skybox reflected on the sphere
3. For help in steps 2-b) and 2-d) you can follow the guide in:
learnopengl.com/Advanced-OpenGL/Cubemaps

Fast Realistic Rendering

OpenGL Introduction